

Contents

□ Binary Search on Answer / Search Space Reduction — Complete Interview Handbook	1
Table of Contents	1
SECTION 1 — Pattern Overview	2
SECTION 2 — Recognition Guide	3
SECTION 3 — Decision Framework	4
SECTION 4 — Why This Pattern Works	4
SECTION 5 — Algorithm Framework	4
SECTION 6 — Time & Space Complexity	5
SECTION 7 — Edge Cases	6
SECTION 8 — Pros & Cons	7
SECTION 9 — Real World Applications	7
SECTION 10 — Interview Strategy	8
SECTION 11 — Pattern Variations	8
SECTION 12 — Comparison With Other Patterns	9
SECTION 13 — Problem Classification	9
SECTION 14 — 30 Interview Problems	10
SECTION 15 — Common Mistakes	13
SECTION 16 — Optimization Techniques	13
SECTION 17 — Multiple Language Templates	13
SECTION 18 — Dry Runs	15
SECTION 19 — Advanced Concepts	16
SECTION 20 — Senior Engineer Insights	16
SECTION 21 — Cheat Sheet	16
SECTION 22 — Revision Notes (5-Minute Review)	17
SECTION 23 — Practice Roadmap	17
SECTION 24 — Memory Tricks	17
SECTION 25 — Final Summary	18

□ **Binary Search on Answer / Search Space Reduction — Complete Interview Handbook**

Pattern #6 of 28 | DSA Pattern Mastery Series Audience: Senior/Staff SWE interviews (Google, Amazon, Meta, Microsoft, Uber, Airbnb, Atlassian, Grab, Careem, Noon, Talabat, Dubai/Saudi & India Tier-1/Tier-2 product companies) **Companion:** Pairs with Striver's A2Z DSA Course — Binary Search section

Table of Contents

1. Pattern Overview
 2. Recognition Guide
 3. Decision Framework
 4. Why This Pattern Works
 5. Algorithm Framework
 6. Time & Space Complexity
 7. Edge Cases
 8. Pros & Cons
 9. Real World Applications
 10. Interview Strategy
 11. Pattern Variations
 12. Comparison With Other Patterns
 13. Problem Classification
 14. 30 Interview Problems
 15. Common Mistakes
 16. Optimization Techniques
 17. Multiple Language Templates
 18. Dry Runs
 19. Advanced Concepts
 20. Senior Engineer Insights
 21. Cheat Sheet
 22. Revision Notes
 23. Practice Roadmap
 24. Memory Tricks
 25. Final Summary
-

SECTION 1 — Pattern Overview

1.1 What Is This Pattern?

Binary Search repeatedly halves a **monotonic search space** to locate a target value or the boundary where a predicate flips from false to true (or true to false), in $O(\log n)$ time. “Binary Search on Answer” generalizes this beyond sorted arrays: instead of searching an array, you search the **space of possible answers** (e.g., “minimum capacity that works,” “maximum speed that’s sufficient”) — as long as that answer space exhibits a monotonic “feasible/infeasible” boundary.

1.2 Why Was This Pattern Invented?

Linear scanning to find a value or boundary costs $O(n)$. If the space has monotonic structure (everything before some point is “no,” everything after is “yes,” or vice versa), you can discard half the remaining space with a single comparison — a direct consequence of the same divide-and-conquer principle behind merge sort, but applied to searching rather than sorting.

1.3 Real Intuition Behind The Pattern

Think of the “guess a number between 1 and 100” game: instead of guessing 1, 2, 3, ... sequentially, you guess 50, and based on “higher/lower” feedback, you eliminate half the remaining possibilities every time — reaching the answer in at most 7 guesses instead of up to 100.

1.4 Mental Model

Binary Search on Answer reframes the problem as: **“Can I verify, in some feasibility-check function `canDo(x)`, whether answer `x` works?”** If `canDo` is monotonic (once true, stays true for all larger/smaller `x`, depending on direction), you binary search over `x` itself, not over an array index.

1.5 Visual Explanation

Classic array search:

```
[1, 3, 5, 7, 9, 11, 13] target=7
mid=7 → FOUND at index 3
```

Binary Search on Answer ("minimum capacity to ship packages in D days"):

```
capacity range: [maxWeight ... totalWeight]
lo=15                               hi=100
mid=57: canShipInDays(57) → check feasibility
if feasible: hi = mid (search smaller capacities)
if not:      lo = mid + 1 (need larger capacity)
... converges to minimum feasible capacity
```

1.6 Simple Analogy

Binary Search on Answer is like **tuning a shower to find the minimum water pressure that still gets hot enough** — you don’t try every pressure setting one by one; you try a middle setting, check if it’s “hot enough” (a monotonic property — higher pressure is at least as hot), and narrow your search range by half each time.

1.7 When Should I Immediately Think About Using This Pattern?

- The array is **sorted** and you need to find a target or an insertion boundary.
- The problem says **“minimum/maximum X such that condition holds”** and you can write a `feasible(x)` check.

- The **answer space is monotonic** — feasibility flips exactly once as X increases (or decreases).
- Constraints suggest $O(\log n)$ or $O(n \log n)$ is expected where $O(n)$ brute force is too slow.

SECTION 2 — Recognition Guide

2.1 Keywords

Keyword	Signal
"sorted array"	Classic binary search
"minimum/maximum ... such that"	Binary Search on Answer
"capacity to ship in D days"	Classic "search the answer" framing
"smallest divisor such that"	Binary Search on Answer
"first/last occurrence"	Boundary binary search (lower_bound/upper_bound)
"peak element"	Binary search on unsorted-but-locally-monotonic data
"koko eating bananas" style (rate/speed minimization)	Binary Search on Answer

2.2 Hidden Hints

Constraints like $1 \leq n \leq 10^9$ (too large to iterate linearly) combined with a phrase like "minimum value such that X is possible" is the single strongest tell for Binary Search on Answer — brute-force iterating every candidate answer would be too slow, but checking one specific candidate's feasibility is fast (often $O(n)$ or $O(n \log n)$), and feasibility is monotonic.

2.3 Interview Clues

Interviewer asks "can you do better than $O(n)$?" after a linear scan on sorted data, or frames the problem as "find the optimal threshold value" rather than "find this element in the array."

2.4 Common Trick Words

"At least," "at most," "smallest such that," "largest such that" — these all imply a monotonic threshold, which is the core requirement for Binary Search on Answer to apply.

2.5 What Interviewers Expect

Correct identification of the monotonic predicate, careful boundary handling ($lo \leq hi$ vs $lo < hi$, mid rounding direction), and a clear articulation of *why* the predicate is monotonic (not just assuming it).

2.6 When NOT To Use This Pattern

- The search space **isn't monotonic** — feasibility flips back and forth as X changes (no clean boundary to binary search over).
- The array is **unsorted and has no exploitable structure** — sort first ($O(n \log n)$) or use a different technique (hashing) if order isn't needed.
- The feasibility check itself is **more expensive than a full brute-force scan** would be — then binary search adds complexity ($O(\log(\text{range})) \times \text{feasibility cost}$) without a net benefit.

SECTION 3 — Decision Framework

Is there a SORTED array and you need to find a target/boundary?

Yes → CLASSIC BINARY SEARCH (search over array indices)

No

Can you define a feasibility check `canDo(x)` for a candidate answer `x`?

Yes

Is `canDo(x)` MONOTONIC (true for all `x` > threshold, false before - or vice versa)?

Yes → BINARY SEARCH ON ANSWER (search over the answer's value range)

No → Binary Search does NOT apply - consider Greedy, DP, or brute force with pruning instead

Why: The entire correctness of binary search — classic or “on answer” — depends on monotonicity. Without it, halving the search space can discard the actual answer, producing silently wrong results. Always verify monotonicity explicitly (mentally or via a quick proof) before applying this pattern.

SECTION 4 — Why This Pattern Works

Mathematical: If $f(x)$ is monotonic (non-decreasing or non-increasing) over the search range, then for any midpoint `mid`, evaluating $f(\text{mid})$ tells you which half of the remaining range can be **safely discarded** — the same elimination-by-domination argument as Two Pointers, but applied along a single value axis instead of two ends of an array. Each iteration halves the search range, so after k iterations the range size is $n / 2^k$; solving $n / 2^k = 1$ gives $k = \log_2(n)$ iterations to converge.

Logical: Classic binary search on a sorted array: if `arr[mid] < target`, every element at index `< mid` is also `< target` (by sortedness), so the entire left half can be discarded without checking it individually.

Intuitive: Every “guess” gives you a full half of the remaining space’s worth of information, not just one data point — an efficient use of the comparison.

Correctness Proof (general “search on answer” form): *Invariant:* the true answer always lies within `[lo, hi]`. *Base case:* `[lo, hi]` initialized to the full valid range — trivially contains the answer. *Inductive step:* evaluating `canDo(mid)` and shrinking to `[lo, mid]` or `[mid+1, hi]` preserves the invariant because monotonicity guarantees the discarded half cannot contain the boundary answer. *Termination:* when `lo == hi`, the invariant collapses to a single point, which must be the answer. **QED.**

SECTION 5 — Algorithm Framework

5.1 Step-by-Step Framework (Classic Binary Search)

1. Initialize `lo = 0, hi = n - 1`.
2. While `lo <= hi`: compute `mid = lo + (hi - lo) / 2`.
3. If `arr[mid] == target`, return `mid`.
4. If `arr[mid] < target`, search right half: `lo = mid + 1`.
5. Else search left half: `hi = mid - 1`.

6. If loop ends without finding, target isn't present.

5.2 Step-by-Step Framework (Binary Search on Answer)

1. Determine the valid answer range [lo, hi] (e.g., lo = 1, hi = max possible value).
2. While lo < hi: compute mid = lo + (hi - lo) / 2.
3. If canDo(mid) is true (feasible): this might be the answer, or a better one exists — narrow toward it: hi = mid (if minimizing) or lo = mid (if maximizing, with careful mid rounding to avoid infinite loops).
4. If canDo(mid) is false: move away: lo = mid + 1 (minimizing) or hi = mid - 1 (maximizing).
5. Return lo (or hi) as the converged answer.

5.3 General Template — Classic Binary Search

```
function binarySearch(arr, target):
    lo, hi = 0, length(arr) - 1
    while lo <= hi:
        mid = lo + (hi - lo) // 2
        if arr[mid] == target: return mid
        elif arr[mid] < target: lo = mid + 1
        else: hi = mid - 1
    return -1
```

5.4 General Template — Binary Search on Answer (Minimize Feasible Value)

```
function binarySearchOnAnswer(lo, hi, canDo):
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if canDo(mid):
            hi = mid                # mid works; try smaller
        else:
            lo = mid + 1            # mid doesn't work; need larger
    return lo                       # smallest value for which canDo is true
```

5.5 Interview Thinking Process

1. "Let me check if the array is sorted, or if I can define a monotonic feasibility function over a range of candidate answers."
2. "I'll state explicitly why this feasibility function is monotonic before coding — this is the correctness backbone."
3. "I'll pick lo and hi bounds carefully — usually the tightest possible valid range to minimize iterations."
4. "I'll dry-run the boundary shrink logic (hi = mid vs hi = mid - 1) on a tiny example to avoid infinite loops or off-by-one errors."
5. "Total complexity: O(log(range)) iterations × cost of canDo per iteration."

SECTION 6 — Time & Space Complexity

Case	Time	Space	Why
Worst Case	$O(\log n)$ classic; $O(\log(\text{range}) \times \text{cost of canDo})$ for search-on-answer	$O(1)$ iterative / $O(\log n)$ recursive (call stack)	Each iteration halves the remaining space
Average Case	Same as worst — deterministic halving regardless of data	$O(1)$	No dependency on data distribution
Best Case	$O(1)$ if target found at first mid	$O(1)$	Still must perform at least one comparison
Amortized	N/A (single search, not batched)	$O(1)$	Each search is independent, not amortized over multiple calls

Why the “canDo cost” matters: total complexity for Binary Search on Answer is $O(\log(\text{range}) \times C)$ where C is the feasibility check’s cost — if `canDo` itself is $O(n)$, total is $O(n \log(\text{range}))$, which is the typical complexity class for these problems (e.g., “Koko Eating Bananas,” “Capacity to Ship Packages”).

SECTION 7 — Edge Cases

Edge Case	Example	Handling
Empty array	<code>[]</code>	Return “not found” immediately
Single element	<code>[5]</code>	<code>lo == hi</code> initially, one comparison resolves it
Target not present	<code>[1, 3, 5]</code> , <code>target=4</code>	Loop terminates with <code>lo > hi</code> , correctly returns “not found”
Duplicate elements (finding first/last occurrence)	<code>[1, 2, 2, 2, 3]</code>	Requires modified binary search that continues narrowing even after finding a match
Answer at the boundary of the range	<code>lo</code> or <code>hi</code> itself is the answer	Verify loop condition (<code>lo < hi</code> vs <code>lo <= hi</code>) doesn’t exclude boundary values
Overflow in mid calculation	Very large <code>lo + hi</code>	Always use <code>lo + (hi - lo) / 2</code> , never <code>(lo + hi) / 2</code> , to avoid integer overflow in languages with fixed-width integers
Non-monotonic predicate misapplied	Feasibility oscillates	Binary search silently gives wrong answer — must verify monotonicity before applying

Edge Case	Example	Handling
Infinite loop from incorrect mid rounding	<code>lo = mid</code> without adjusting rounding	When narrowing with <code>lo = mid</code> (not <code>mid+1</code>), must round <code>mid</code> up (<code>mid = lo + (hi-lo+1)/2</code>) to guarantee progress

Common mistakes: using $(lo+hi)/2$ (overflow risk in languages like Java/C++ with fixed-width integers); infinite loops when `lo = mid` doesn't advance because of incorrect rounding direction; assuming monotonicity without verifying it.

SECTION 8 — Pros & Cons

Advantages: $O(\log n)$ or $O(\log(\text{range}))$ — exponentially faster than linear scanning; generalizes far beyond arrays to any monotonic decision problem (“minimize/maximize X such that...”). **Disadvantages:** Requires provable monotonicity — misapplying it to non-monotonic problems gives silently wrong answers, not a crash; boundary/rounding bugs are common and subtle. **Trade-offs:** Binary Search on Answer ($O(\log(\text{range}) \times C)$) vs. brute-force linear scan over candidate answers ($O(\text{range} \times C)$) — always prefer binary search when monotonicity holds and range is large. **Limitations:** Doesn't work on unsorted data without an exploitable monotonic structure; doesn't directly solve problems where the “feasible” set isn't contiguous. **Inefficient when:** the feasibility check `canDo(x)` itself is asymptotically expensive relative to just brute-forcing all candidates directly (rare, but possible in exotic problems).

SECTION 9 — Real World Applications

Domain	Application
Google	Search engine index lookups (sorted posting lists), and capacity-planning algorithms (minimum resources needed for SLA)
Amazon	Warehouse/shipping capacity planning (“minimum truck capacity to ship all packages within D days” — literally a canonical LeetCode problem based on real logistics)
Meta	A/B test threshold tuning — finding the minimum sample size or threshold satisfying a statistical power requirement
Netflix	Adaptive bitrate selection — finding the minimum bitrate that satisfies a buffering constraint given network conditions
Uber	Dynamic pricing threshold search — finding the minimum surge multiplier that balances supply and demand
Databases	B-Tree index lookups are essentially binary search generalized to disk-block-sized nodes
Operating Systems	Binary search used in kernel memory allocators for finding suitable free blocks in sorted free-lists

Domain	Application
Networking	Binary search used in TCP-like congestion window tuning (finding maximum sustainable throughput)
Version Control	<code>git bisect</code> — literally binary search over commit history to find the commit that introduced a bug
Compilers/Systems	Binary search over configuration parameters (e.g., minimum thread pool size satisfying throughput requirements) during performance tuning

SECTION 10 — Interview Strategy

How seniors answer: They explicitly state the monotonicity assumption and prove it in one sentence (“if capacity C works, any capacity $> C$ also works, since more capacity can only help — so feasibility is monotonic”) before writing any code, then carefully choose the shrink direction and verify no infinite loop is possible.

How juniors answer: They often apply the “binary search template” mechanically without verifying monotonicity, or they get the $lo = mid$ vs $lo = mid + 1$ boundary wrong, leading to infinite loops that they then patch reactively without understanding the root cause.

Typical follow-ups: “How do you know this predicate is monotonic?” “What if there are duplicates — how do you find first/last occurrence?” “Can you avoid the overflow risk in mid calculation?” “How would you extend this to a 2D sorted matrix?”

Optimization questions: “Can you reduce the feasibility check’s cost from $O(n)$ to $O(\log n)$?” (sometimes possible with auxiliary precomputation like prefix sums).

SECTION 11 — Pattern Variations

Variation	Description	Example
Classic Binary Search	Find exact target in sorted array	Binary Search (LeetCode 704)
Lower Bound / Upper Bound	Find first/last position satisfying a condition	Find First and Last Position of Element in Sorted Array
Binary Search on Answer (Minimize)	Find smallest feasible value	Koko Eating Bananas, Capacity To Ship Packages
Binary Search on Answer (Maximize)	Find largest feasible value	Split Array Largest Sum (minimize max, related), Aggressive Cows (maximize min distance)
Binary Search on Rotated Array	Modified comparison logic for rotated sorted arrays	Search in Rotated Sorted Array
Binary Search on 2D Matrix	Treat 2D sorted matrix as a flattened 1D search space	Search a 2D Matrix

Variation	Description	Example
Binary Search for Peak Element	Search using local monotonicity (slope direction), not global sortedness	Find Peak Element

SECTION 12 — Comparison With Other Patterns

Pattern	Difference	When To Prefer
Two Pointers	Searches for pairs/triplets via elimination from both ends	Pair/triplet search, not single-value threshold search
Sliding Window	Tracks a contiguous range's running state	Contiguous subrange optimization, not global value search
Greedy	Makes locally optimal choices without a "feasibility" framing	When no clean monotonic threshold exists but local optimality still leads to a global optimum
Dynamic Programming	Explores overlapping subproblems, often when the answer space isn't simply monotonic	Non-monotonic or combinatorial optimization

Comparison Table

Aspect	Binary Search on Answer	Linear Scan	Dynamic Programming
Requires monotonicity	Yes	No	No
Time complexity	$O(\log(\text{range}) \times C)$	$O(\text{range} \times C)$	Varies (often $O(n^2)$ or more)
Best for	Single monotonic threshold optimization	Small ranges, no structure	Combinatorial/non-monotonic optimization

SECTION 13 — Problem Classification

Difficulty	Characteristics	Examples
Easy	Direct sorted-array search	Binary Search, Search Insert Position
Medium	Boundary search, rotated array search	Find First and Last Position, Search in Rotated Sorted Array

Difficulty	Characteristics	Examples
Hard	Binary Search on Answer with a non-trivial feasibility function	Koko Eating Bananas, Capacity To Ship Packages Within D Days, Split Array Largest Sum
Very Hard	Binary search combined with additional structures (2D, merge, or DP-hybrid feasibility)	Median of Two Sorted Arrays, Aggressive Cows (advanced framing), Minimize Max Distance to Gas Station

SECTION 14 — 30 Interview Problems

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
1	Binary Search	Easy	Amazon, Google, Microsoft	Classic template	Foundational mechanics
2	Search Insert Position	Easy	Amazon, Meta	Boundary/lower bound search	Insertion point logic
3	Find First and Last Position of Element in Sorted Array	Medium	Amazon, Meta, Microsoft	Lower bound + upper bound search	Duplicate handling
4	Search in Rotated Sorted Array	Medium	Amazon, Meta, Microsoft, Google	Modified monotonicity in rotated array	Adapting binary search to broken sortedness
5	Find Minimum in Rotated Sorted Array	Medium	Amazon, Meta	Modified monotonicity	Rotated array boundary detection
6	Find Peak Element	Medium	Google, Amazon	Local monotonicity (slope-based)	Non-globally-sorted binary search
7	Search a 2D Matrix	Medium	Amazon, Microsoft	Flattened 2D binary search	Dimensional extension
8	Search a 2D Matrix II	Medium	Amazon, Meta	Staircase search (related but distinct technique)	Contrast with pure binary search
9	Koko Eating Bananas	Medium	Google, Amazon, Meta	Classic Binary Search on Answer (minimize speed)	Core “search on answer” mechanics

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
10	Capacity To Ship Packages Within D Days	Medium	Amazon, Google	Binary Search on Answer (minimize capacity)	Feasibility function design
11	Split Array Largest Sum	Hard	Google, Amazon, Meta	Binary Search on Answer (minimize max subarray sum)	Advanced feasibility function
12	Minimize Max Distance to Gas Station	Hard	Google	Binary Search on Answer with floating-point precision	Continuous answer space
13	Aggressive Cows (GFG/Codeforces classic)	Hard	Google, Amazon (conceptually asked)	Binary Search on Answer (maximize min distance)	Maximize-the-minimum framing
14	Median of Two Sorted Arrays	Hard	Google, Amazon, Meta, Microsoft	Binary search over partition point	Advanced partition-based binary search
15	Kth Smallest Element in a Sorted Matrix	Medium	Amazon, Google	Binary Search on Answer (value space, not index space)	Value-space binary search
16	Find the Smallest Divisor Given a Threshold	Medium	Google	Binary Search on Answer (minimize divisor)	Feasibility function design
17	Magnetic Force Between Two Balls	Medium	Google, Amazon	Binary Search on Answer (maximize min distance)	Maximize-the-minimum framing
18	Minimum Number of Days to Make m Bouquets	Medium	Amazon	Binary Search on Answer (minimize days)	Feasibility with grouping constraint
19	Divide Chocolate	Hard	Google	Binary Search on Answer (maximize min piece)	Maximize-the-minimum framing

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
20	Sqrt(x)	Easy	Amazon, Microsoft	Binary search over value space	Basic value-space search
21	Single Element in a Sorted Array	Medium	Amazon, Google	Parity-based binary search	Non-standard monotonicity exploitation
22	Find K-th Smallest Pair Distance	Hard	Google	Binary Search on Answer + two-pointer feasibility check	Cross-pattern combination
23	Allocate Minimum Number of Pages (GFG classic)	Hard	Amazon, Microsoft (conceptually asked)	Binary Search on Answer (minimize max pages)	Feasibility function design
24	Time Based Key-Value Store	Medium	Amazon, Meta	Binary search on timestamp-sorted values	Applied binary search in system design context
25	Search Suggestions System	Medium	Amazon, Meta	Binary search combined with sorting/prefix matching	Hybrid application
26	Find Right Interval	Medium	Google	Binary search on sorted interval starts	Interval-based binary search
27	Count of Smaller Numbers After Self	Hard	Google, Amazon	Binary search combined with BIT/merge sort	Advanced hybrid counting
28	Nth Digit	Medium	Google	Binary search over digit-length boundaries	Mathematical binary search framing
29	Random Pick with Weight	Medium	Amazon, Meta	Binary search over cumulative weight (prefix sum + binary search)	Cross-pattern combination

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
30	Minimum Limit of Balls in a Bag	Medium	Google, Amazon	Binary Search on Answer (minimize max penalty)	Feasibility function design

SECTION 15 — Common Mistakes

1. Using $(lo + hi) / 2$ instead of $lo + (hi - lo) / 2$, risking integer overflow in fixed-width-integer languages. *Fix:* always use the subtraction form.
2. Infinite loops from incorrect narrowing when using $lo = mid$ (not $mid + 1$) without adjusting the mid-rounding direction. *Fix:* when narrowing upward with $lo = mid$, round mid up: $mid = lo + (hi - lo + 1) / 2$.
3. Applying binary search without verifying monotonicity — produces silently wrong (not crashing) results. *Fix:* always state the monotonicity proof explicitly before coding.
4. Off-by-one in loop condition ($lo \leq hi$ for classic search vs $lo < hi$ for “search on answer” convergence). *Fix:* understand which convention matches which problem shape and be consistent.
5. Forgetting to validate whether the answer even exists within the initial $[lo, hi]$ bounds before starting the search. *Fix:* explicitly check the feasibility of hi (or lo) upfront if “no solution” is a valid outcome.

Why people fail: binary search’s logic is simple, but its correctness is entirely dependent on an *assumption* (monotonicity) that isn’t always stated in the problem — candidates who don’t explicitly verify this assumption sometimes apply the pattern to problems where it silently fails, which is far more dangerous than a crash because the wrong answer looks plausible.

SECTION 16 — Optimization Techniques

- **Time:** Tighten the initial $[lo, hi]$ range as much as possible (e.g., $lo = \max(weights)$, not $lo = 0$) to reduce iteration count.
- **Space:** Prefer iterative over recursive binary search to avoid $O(\log n)$ call-stack overhead, though this is rarely a bottleneck in practice.
- **Readability:** Extract the feasibility check into a clearly named helper function (`canShipInDays(capacity)`), not inline logic, to make the monotonicity argument explicit in the code structure itself.
- **Interview performance:** Explicitly state the monotonicity proof and the exact narrowing direction ($hi = mid$ vs $lo = mid + 1$) before writing code — this is the single highest-leverage habit for this pattern.

SECTION 17 — Multiple Language Templates

Java

```
public int minEatingSpeed(int[] piles, int h) {
    int lo = 1, hi = Arrays.stream(piles).max().getAsInt();
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
```

```

        if (canFinish(piles, h, mid)) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}
private boolean canFinish(int[] piles, int h, int speed) {
    long hours = 0;
    for (int p : piles) hours += (p + speed - 1) / speed;
    return hours <= h;
}

```

JavaScript

```

function minEatingSpeed(piles, h) {
    let lo = 1, hi = Math.max(...piles);
    const canFinish = (speed) => piles.reduce((acc, p) => acc + Math.ceil(p / speed), 0) <= h;
    while (lo < hi) {
        const mid = lo + Math.floor((hi - lo) / 2);
        if (canFinish(mid)) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}

```

PHP

```

function minEatingSpeed(array $piles, int $h): int {
    $lo = 1; $hi = max($piles);
    $canFinish = function($speed) use ($piles, $h) {
        $hours = 0;
        foreach ($piles as $p) $hours += (int)ceil($p / $speed);
        return $hours <= $h;
    };
    while ($lo < $hi) {
        $mid = $lo + intval(($hi - $lo) / 2);
        if ($canFinish($mid)) $hi = $mid;
        else $lo = $mid + 1;
    }
    return $lo;
}

```

Python

```

import math
def min_eating_speed(piles, h):
    lo, hi = 1, max(piles)
    def can_finish(speed):
        return sum(math.ceil(p / speed) for p in piles) <= h
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if can_finish(mid):
            hi = mid
        else:

```

```

        lo = mid + 1
    return lo

```

Go

```

func minEatingSpeed(piles []int, h int) int {
    lo, hi := 1, 0
    for _, p := range piles {
        if p > hi { hi = p }
    }
    canFinish := func(speed int) bool {
        hours := 0
        for _, p := range piles {
            hours += (p + speed - 1) / speed
        }
        return hours <= h
    }
    for lo < hi {
        mid := lo + (hi-lo)/2
        if canFinish(mid) {
            hi = mid
        } else {
            lo = mid + 1
        }
    }
    return lo
}

```

C++

```

int minEatingSpeed(vector<int>& piles, int h) {
    int lo = 1, hi = *max_element(piles.begin(), piles.end());
    auto canFinish = [&](int speed) {
        long hours = 0;
        for (int p : piles) hours += (p + speed - 1) / speed;
        return hours <= h;
    };
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (canFinish(mid)) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}

```

SECTION 18 — Dry Runs

Small Input

piles = [3, 6, 7, 11], h = 8 (Koko Eating Bananas)

lo=1, hi=11

mid=6: hours = ceil(3/6)+ceil(6/6)+ceil(7/6)+ceil(11/6) = 1+1+2+2 = 6 <= 8 → feasible → hi=6

```

mid=3:  hours = 1+2+3+4 = 10 > 8 → not feasible → lo=4
mid=5:  hours = 1+2+2+3 = 8 ≤ 8 → feasible → hi=5
mid=4:  hours = 1+2+2+3 = 8 ≤ 8 → feasible → hi=4
lo=4, hi=4 → converged. Answer = 4

```

Large Input (Conceptual)

For piles with values up to 10^9 and h up to 10^9 , the answer range is up to 10^9 , requiring only ~ 30 iterations of binary search ($\log_2(10^9) \approx 30$), each costing $O(n)$ for the feasibility check — total $O(30n)$, vastly better than brute-force checking every possible speed from 1 to 10^9 .

Corner Case

piles = [1], $h = 1$: $lo=1, hi=1 \rightarrow$ loop doesn't execute ($lo == hi$ already) $\rightarrow$ answer = 1 directly, correctly handling the trivial single-pile case.

SECTION 19 — Advanced Concepts

- **Partition-based binary search (Median of Two Sorted Arrays):** instead of searching for a value, binary search over the **partition point** in the smaller array such that the combined left partition and right partition satisfy a size/value balance — a significantly more advanced application of the same halving principle.
- **Binary search on floating-point ranges:** for continuous answer spaces (e.g., minimize maximum distance with real-valued positions), loop a fixed number of iterations (e.g., 100) or until $hi - lo < \epsilon$, rather than using integer equality.
- **Binary search combined with greedy feasibility check:** many “Binary Search on Answer” problems use a **greedy simulation** as the `canDo(x)` check (e.g., “can we ship all packages with capacity C in D days” is checked greedily by simulating loading day by day) — recognizing this composition (Binary Search + Greedy) is a hallmark of Hard-level problems.
- **Parametric search:** the formal name for the general technique of “turn an optimization problem into a decision problem, then binary search over the decision” — useful vocabulary to demonstrate CS fundamentals depth in interviews.

SECTION 20 — Senior Engineer Insights

Staff engineers recognize Binary Search on Answer as an instance of **parametric search** — transforming “find the optimal X ” into “can I verify a candidate X quickly, and is verification monotonic in X ?” This reframing shows up far beyond LeetCode: capacity planning (minimum server count satisfying an SLA), pricing algorithms (minimum price satisfying demand constraints), and resource allocation (minimum budget satisfying a coverage requirement) all reduce to the same shape. Interviewers evaluate whether a candidate can perform this transformation **unprompted** on a novel problem that doesn't superficially resemble “search an array,” which is the true test of understanding versus template memorization.

SECTION 21 — Cheat Sheet

PATTERN: Binary Search / Binary Search on Answer

RECOGNIZE: sorted array search, OR “minimum/maximum X such that `condition(X)`” with monotonic feasibility

TEMPLATE (classic):

```
lo, hi = 0, n-1
```

```
while lo <= hi: mid = lo + (hi-lo)//2; compare arr[mid] to target; shrink accordingly
```


TEMPLATE (on answer, minimize):

```
lo, hi = minPossible, maxPossible
while lo < hi: mid = lo + (hi-lo)//2; if canDo(mid): hi = mid; else: lo = mid + 1
return lo
```

COMPLEXITY: $O(\log n)$ classic; $O(\log(\text{range}) \times \text{cost of canDo})$ for search-on-answer

KEY PROOF: monotonicity of the predicate guarantees safe elimination of half the space each iteration

WATCH FOR: overflow in mid calc, infinite loops from wrong narrowing direction, unverified monotonicity

DOESN'T APPLY WHEN: predicate isn't monotonic, data has no exploitable order

SECTION 22 — Revision Notes (5-Minute Review)

- Binary Search halves a monotonic search space every iteration — $O(\log n)$ or $O(\log(\text{range}))$.
 - “On Answer” variant: define `canDo(x)`, verify it’s monotonic, binary search over the value range, not array indices.
 - Always use `lo + (hi-lo)/2` to avoid overflow.
 - Narrowing convention matters: `lo = mid+1/hi = mid` for minimize; adjust mid-rounding for maximize to avoid infinite loops.
 - Total cost = $O(\log(\text{range}) \times \text{cost of feasibility check})$ — often the feasibility check itself is $O(n)$, giving $O(n \log(\text{range}))$ overall.
 - Never apply without first proving/stating monotonicity — this is the #1 senior-vs-junior differentiator.
-

SECTION 23 — Practice Roadmap

Stage	Focus	Recommended LeetCode Problems
Beginner	Classic binary search mechanics	Binary Search (704), Search Insert Position (35), Sqrt(x) (69)
Intermediate	Boundary search, rotated arrays	Find First and Last Position (34), Search in Rotated Sorted Array (33), Find Peak Element (162)
Advanced	Binary Search on Answer	Koko Eating Bananas (875), Capacity To Ship Packages Within D Days (1011), Find the Smallest Divisor Given a Threshold (1283)
Expert	Advanced feasibility functions, partition-based search	Split Array Largest Sum (410), Median of Two Sorted Arrays (4), Divide Chocolate (1231)

SECTION 24 — Memory Tricks

- **Mnemonic:** “**H**alve, **C**heck, **E**liminate” — Halve the range, Check the midpoint, Eliminate the impossible half.
- **Visualization:** The “**guess the number**” game — every guess should eliminate half the remaining possibilities, not just one number.

- **Recognition shortcut:** “Minimum/maximum X such that...” + a range too large to brute-force → Binary Search on Answer, define `canDo(x)` first.
-

SECTION 25 — Final Summary

Binary Search — whether classic array search or the generalized “search on answer” form — converts $O(n)$ or $O(\text{range})$ linear exploration into $O(\log n)$ or $O(\log(\text{range}))$ by repeatedly halving a **provably monotonic** search space. The single most important thing to remember forever: **binary search’s correctness depends entirely on monotonicity — before applying it to any new problem, explicitly state why “if X works, everything past X in the same direction also works,” and only then binary search over that value.** The generalization from “search an array” to “search any monotonic answer space” (parametric search) is what separates senior-level pattern mastery from junior-level template memorization.